Frequent Itemsets 1

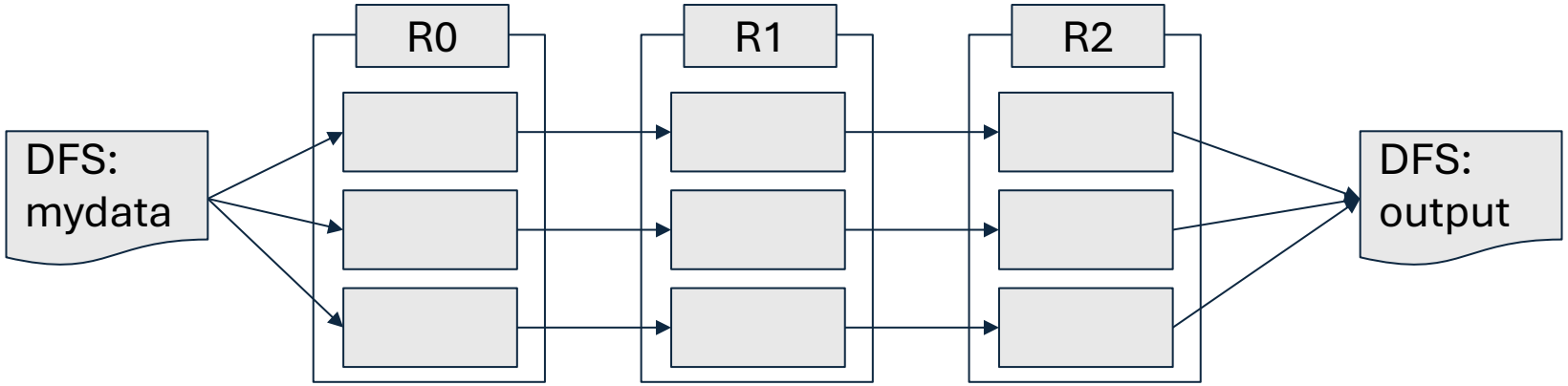
Introduction to Data Mining

Seoul National University

Spring 2026

Recap: Spark

- Spark generalizes MapReduce with a DAG of operations.
  - **Data abstraction:** Resilient distributed datasets (RDDs).
  - **Operations:** Transformations (e.g., map) and actions (e.g., reduce).
  - **Two improvements:** Lazy evaluation and the lineage of RDDs.



Outline

1. **<u>Frequent Itemsets</u>**
2. Association Rules
3. Finding Frequent Pairs
4. A-Priori Algorithm

Managing a Supermarket

- In a supermarket, the manager wants to find **patterns** of items.
- If the dataset is huge, it will take forever to do it manually.

Source: Wikimedia Commons

Source: Needpix.com

Association Rule Discovery

- **Goal:** Identify items that are bought together by many customers.
- **Approach:** Process sales data to find *dependencies* among items.
- **A classic rule:**
  - If someone buys diaper and milk, then he/she is likely to buy beer.
  - Don't be surprised if you find six-packs next to diapers!
- **Association rules:** People who bought $\{x, y\}$ tend to buy $\{z, v\}$.

The Market-Basket Model

- **Market-basket model** is a many-to-many mapping between:
  - A large set of **items** (e.g., things sold in a supermarket)
  - A large set of **baskets** (e.g., things a customer buys at one time)
    - Each basket is a small subset of items.

Basket	*Items*
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Coke, Diaper, Milk

Source: Stanford CS246 (2022)

Frequent Itemsets

- **Goal:** Find sets of items that appear together frequently.
- **Support** for itemset I: # of baskets containing all elements in I.
  - Given a support threshold s,
  - Itemset I is a **frequent itemset** if it appears at least in s baskets.

Basket	*Items*
1	Bread, Coke, Milk
2	Beer, Bread
3	Beer, Coke, Diaper, Milk
4	Beer, Bread, Diaper, Milk
5	Coke, Diaper, Milk

Source: Stanford CS246 (2022)

Support of {Beer, Bread} is 2

Example

- Items = {milk, coke, pepsi, beer, juice}
- Support threshold = 3 baskets

B1 = {m, c, b}	B2 = {m, p, j}
B3 = {m, b}	B4 = {c, j}
B5 = {m, p, b}	B6 = {m, c, b, j}
B7 = {c, b, j}	B8 = {b, c}

Example

- Items = {milk, coke, pepsi, beer, juice}
- Support threshold = 3 baskets

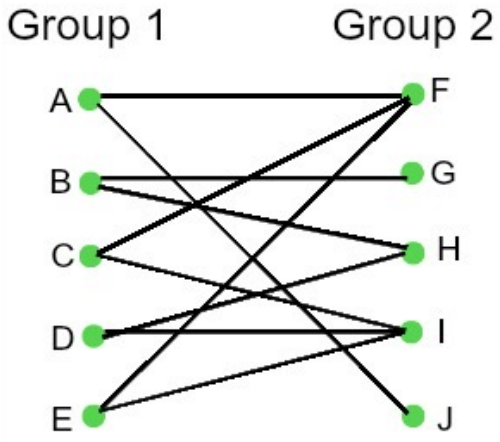
B1 = {m, c, b}	B2 = {m, p, j}
B3 = {m, b}	B4 = {c, j}
B5 = {m, p, b}	B6 = {m, c, b, j}
B7 = {c, b, j}	B8 = {b, c}

- Frequent itemsets:

  {m}, {c}, {b}, {j}, {m, b}, {b, c}, {c, j}

Market-Basket Model as Abstract

- Market-basket model is an **abstract** of various data.
  - **Topic discovery:** items = words; baskets = documents
- Items do not have to be "in" baskets!
  - As long as a dataset is a many-to-many mapping.
  - **Plagiarism:** items = documents; baskets = sentences
  - **Biomarkers:** items = drugs; baskets = patients



Source: Study.com

Outline

1. Frequent Itemsets
2. **<u>Association Rules</u>**
3. Finding Frequent Pairs
4. A-Priori Algorithm

Association Rules

- **Given** Itemsets I and J,
- **Find** an **association rule** $I \rightarrow J$ such that if a basket contains all of the items in I, then it is likely to contain J as well.
  - $I \cap J = \emptyset$ is assumed by default.

B1 = {m, c, b}	B2 = {m, p, j}
B3 = {m, b}	B4 = {c, j}
B5 = {m, p, b}	B6 = {m, c, b, j}
B7 = {c, b, j}	B8 = {b, c}

Association rule: {m, b} → c

Support

- **Q:** How can we decide if an association rule is *significant*?
- **Support** of $I \rightarrow J$ is defined to be the support of $I \cup J$.
  - $\text{support}(I \rightarrow J) = \text{support}(I \cup J)$.
  - Higher support is better, as in frequent itemsets.

B1 = {m, c, b}	B2 = {m, p, j}	Support({m, b, c}): 2
B3 = {m, b}	B4 = {c, j}	
B5 = {m, p, b}	B6 = {m, c, b, j}	
B7 = {c, b, j}	B8 = {b, c}	

Association rule: {m, b} → c

Confidence

- **Q:** How can we decide if an association rule is *significant*?
- **Confidence** of $I \rightarrow J$ = the probability of J given I in a basket.
  - $\text{conf}(I \rightarrow J) = P(J|I) = \text{P}(I \cup J) \,/\, \text{P}(I)$
  - $\text{conf}(I \rightarrow J) = \text{support}(I \cup J) \,/\, \text{support}(I)$

B1 = {m, c, b}	B2 = {m, p, j}	Support({m, b, c}): 2
B3 = {m, b}	B4 = {c, j}	Support({m, b}): 4
B5 = {m, p, b}	B6 = {m, c, b, j}	Confidence: 2/4 = 0.5
B7 = {c, b, j}	B8 = {b, c}	

Association rule: {m, b} → c

Interest

- Not all high-confidence rules are interesting.
  - The rule X → milk will have high confidence if milk is purchased a lot.
- **Interest** of $I \rightarrow J = \text{conf}(I \rightarrow J) - P(J) \ = P(J|I) - P(J)$.
  - $P(J)$: Fraction of baskets containing J in the dataset.

B1 = {m, c, b}	B2 = {m, p, j}	Support({m, b, c}): 2
B3 = {m, b}	B4 = {c, j}	Support({m, b}): 4
B5 = {m, p, b}	B6 = {m, c, b, j}	Confidence: 2/4 = 0.5
B7 = {c, b, j}	B8 = {b, c}	Interest : 0.5 - 5/8 = -1/8 (negative value?)

Association rule: {m, b} → c

Mining Association Rules

- **Goal:** Find all rules with support $\geq s$ and confidence $\geq c$.
  - Interest is a relative measure, not very suitable as a threshold.
- **Difficult part:** Finding the frequent itemsets.
  - Identifying association rules is easy if we have frequent itemsets.
  - We will study the easy part first.

Mining Association Rules

- **Step 1:** Find all frequent itemsets I (will explain this later).
- **Step 2:** For every subset $A \subset I$, generate a rule $A \to I \setminus A$.
  - **Version 1:** Compute the confidence of every possible rule.
    - $\text{conf}(\{a, b\} \to \{c, d\}) = \text{support}(\{a, b, c, d\}) \,/\, \text{support}(\{a, b\})$.
  - **Version 2:** Eliminate stronger rules by starting from weaker ones.
    - If $\{a, b, c\} \to \{d\}$ is below confidence, then so is $\{a, b\} \to \{c, d\}$.
  - Output the rules above the confidence threshold.

Mining Association Rules

- **Example**
  - Support threshold $s = 3$
  - Confidence threshold $c = 3/4$

B1 = {m, c, b}
B2 = {m, p, j}
B3 = {m, c, b, n}
B4 = {c, j}
B5 = {m, p, b}
B6 = {m, c, b, j}
B7 = {c, b, j}
B8 = {b, c}

1. Frequent itemsets:
{b,m}, {b,c}, {c,m}, {c,j}, {m,c,b}

Mining Association Rules

- **Example**
  - Support threshold $s = 3$
  - Confidence threshold $c = 3/4$

B1 = {m, c, b}
B2 = {m, p, j}
B3 = {m, c, b, n}
B4 = {c, j}
B5 = {m, p, b}
B6 = {m, c, b, j}
B7 = {c, b, j}
B8 = {b, c}

1. Frequent itemsets:
{b,m}, {b,c}, {c,m}, {c,j}, {m,c,b}

2. Generated rules:
{b, c} → m: conf = 3/5
{b, m} → c: conf = 3/4
{m, c} → b: conf = 3/3
b → {c, m}: conf = 3/6
...

Mining Association Rules

- **Example**
  - Support threshold $s = 3$
  - Confidence threshold $c = 3/4$

B1 = {m, c, b}
B2 = {m, p, j}
B3 = {m, c, b, n}
B4 = {c, j}
B5 = {m, p, b}
B6 = {m, c, b, j}
B7 = {c, b, j}
B8 = {b, c}

1. Frequent itemsets:
{b,m}, {b,c}, {c,m}, {c,j}, {m,c,b}

2. Generated rules:
~~{b, c} → m: conf = 3/5~~
{b, m} → c: conf = 3/4
{m, c} → b: conf = 3/3
~~b → {c, m}: conf = 3/6~~
…

Compacting the Output

- Finding a million rules is not meaningful in practice.
- **Q:** What can we do it other than adjusting the thresholds?
- **A:** Post-process frequent itemsets before finding rules.
  - Suppose that $\text{support}(\{A, B, C\}) = \text{support}(\{A, B\})$.
  - Then, we can safely remove $\{A, B\}$ from our candidates.
  - $\text{conf}(\{A\} \rightarrow \{B, C\}) = \text{conf}(\{A\} \rightarrow \{B\})$
  - $\text{conf}(\{B\} \rightarrow \{A, C\}) = \text{conf}(\{B\} \rightarrow \{A\})$

Compacting the Output

- We can retain only the closed / maximal frequent itemsets.
- **Closed itemsets:**
  - No immediate superset has the same support (> 0).
  - **Pros:** Safe way to compact the output.
  - **Cons:** Still numerous and complex to compute.
- **Maximal frequent itemsets:**
  - No immediate superset is frequent.
  - **Pros:** Gives more pruning while being fast.
  - **Cons:** Support details are missing.

Example: Maximal / Closed Itemsets

- **Q:** Why are maximal frequent itemsets not always better?
- **A:** If you store only the maximal set {A,B,C}, you lose that:
  - {A} is extremely common.
  - {A,B} is moderately common.
  - {A,B,C} is barely frequent.

Example: Maximal / Closed Itemsets

- **Example** (s=3)

	Support	Maximal	Closed	
A	4	No	No	
B	5	No	Yes	BC is freq.
C	3	No	No	support(BC) = 3
AB	4	Yes	Yes	ABC is not freq.
AC	2	No	No	
BC	3	Yes	Yes	support(ABC) < 3
ABC	2	No	Yes	

Pop Quiz

- We have the following baskets with support threshold $s = 2$:

  B1 = {apple, butter, milk}

  B2 = {apple, butter, banana, walnuts, milk}

  B3 = {butter, milk, banana}

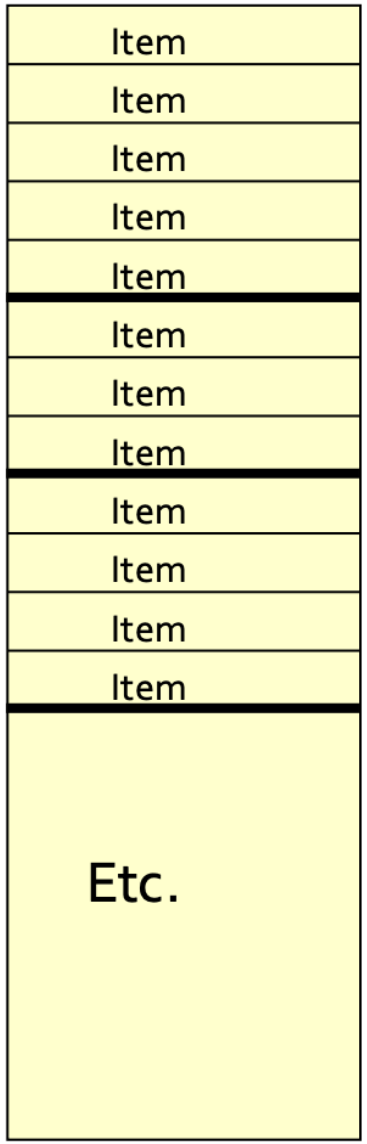
  B4 = {apple, milk}
- **Q1.** Compute the support for itemset {apple, butter}.
- **Q2.** Compute the confidence for the rule {apple} -> {butter}.
- **Q3.** Compute the interest of {apple} -> {butter}.
- **Q4.** Is {apple, butter} maximal and/or closed?

Outline

1. Frequent Itemsets
2. Association Rules
3. **<u>Finding Frequent Pairs</u>**
4. A-Priori Algorithm

Representation of Market-Basket Data

- Let's go back to finding frequent itemsets.
- Market-basket data is stored on disk basket-by-basket.
  - Either in a DFS (baskets are objects in files) or as a file.
- The data can be too large to fit in main memory.
  - Baskets are small but we have many baskets and items.



Source: Stanford CS246 (2022)

Computational Cost

- The true cost of mining large data is usually # of disk I/Os.
- In practice, association-rule algorithms read the data in **passes**.
- We measure the cost by **# of passes** that an algorithm makes.

Main-Memory Bottleneck

- **Main memory** is the issue for frequent-itemset algorithms.
  - As we read baskets, we need to count something, e.g., item occurrences.
  - The number of different things we can count is limited by main memory.
  - Swapping counts in/out is a disaster.
- **Minor technique:** We represent items simply by numbers.
  - By consecutive integers from 1 to n where n is # of distinct items.
  - We can construct a hash table that translates items to integers.

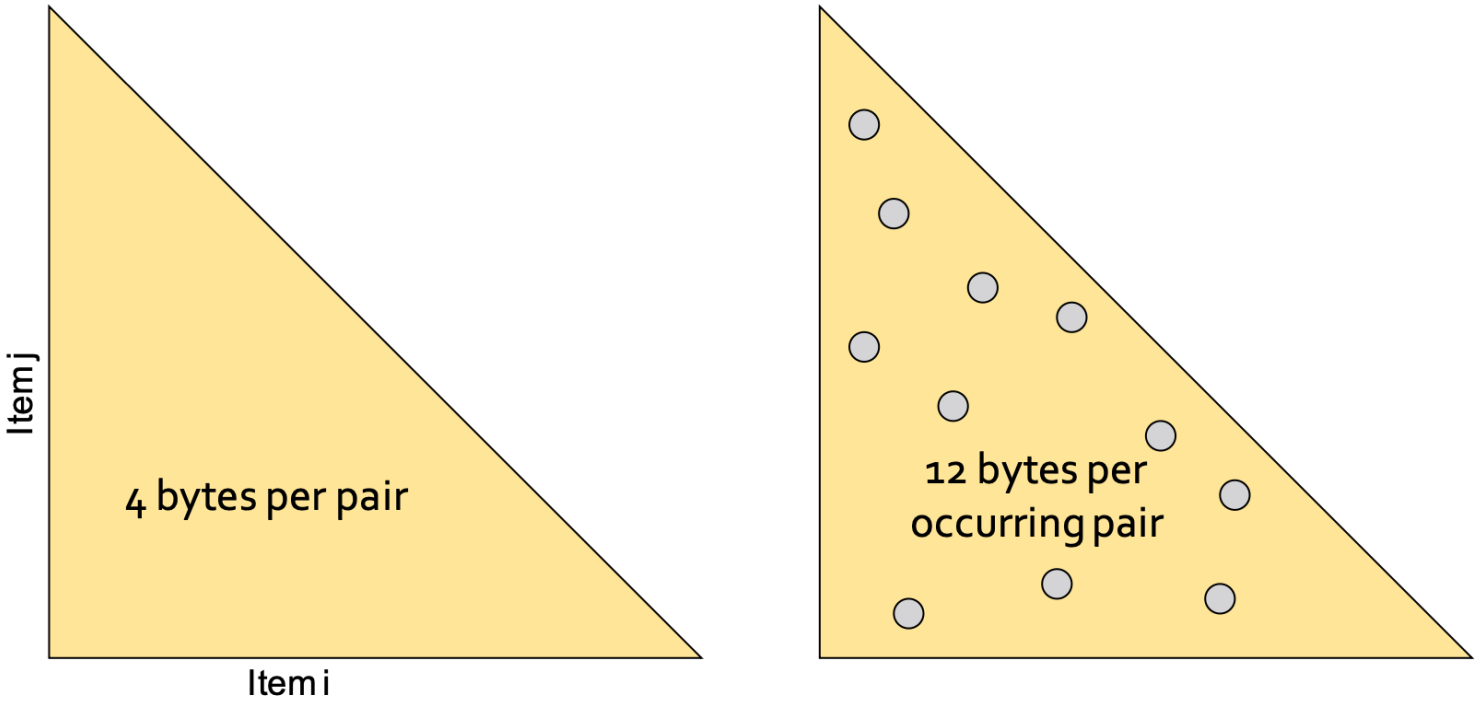
Finding Frequent Pairs

- The hardest problem is finding **frequent pairs** of items (i, j).
  - **Why?** Frequent pairs are common, frequent triples are rare.
  - Probability of being frequent drops exponentially with size.
- Let's first concentrate on pairs, and then extend to larger sets.

Counting Pairs in Memory

- **Goal:** Count the # of occurrences of each pair of items (i, j).
- **Triangular-matrix method:**
  - Use a large matrix (i.e., an array) to store all counts.
- **Triples method:**
  - Keep a hash table of triples (i, j, c), storing count c for each pair (i, j).
  - If integers are 4 bytes, we need about 12 bytes for each pair with $c > 0$.
  - Plus some additional overhead for the hash table.

Comparison of Two Approaches



Triangular Matrix

Triples (item i, item j, count)

Source: Stanford CS246 (2022)

Comparison of Two Approaches

- **Triangular-matrix method:**
  - Counts a pair of items (i, j) only if $i < j$.
  - Pairs are stored in lexicographic order:
    - {1,2}, {1,3}, …, {1,n}, {2,3}, {2,4}, …,{2,n}, … {n-1,n}
  - Total bytes = $4 \times n(n-1)/2$.
- **Triples Method:**
  - Uses 12 bytes per pair only if count > 0.
  - Total bytes = $12 \times p \times n(n-1)/2$.
- The triples method is better if $p < 1/3$.

Naïve Algorithm

- Count the occurrence of each pair in main memory.
  - From each basket b of n_b items, generate its $n_b(n_b - 1)/2$ pairs.
- Fails if (# of items)^2 exceeds main memory.
  - **Remember:** # of items can be 100K (Wal-Mart) or 10B (Web pages).
    - If we have 1M items, 2 terabytes of memory is needed.
    - What if we have 1B items?
- **Q:** How can we do better when we have too many items?

Outline

1. Frequent Itemsets
2. Association Rules
3. Finding Frequent Pairs
4. **<u>A-Priori Algorithm</u>**

A-Priori Algorithm

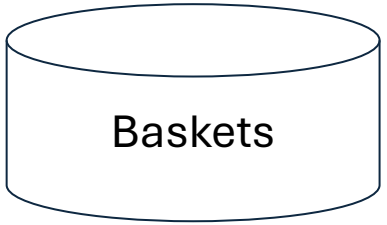
- **A-Priori** is a two-pass approach which limits memory usage.
- Key idea is **monotonicity**.
  - If an itemset I appears at least s times, so does every subset $J \subseteq I$.
- **Contrapositive for pairs:**
  - If item i does not appear in s baskets, then no pair including i can.

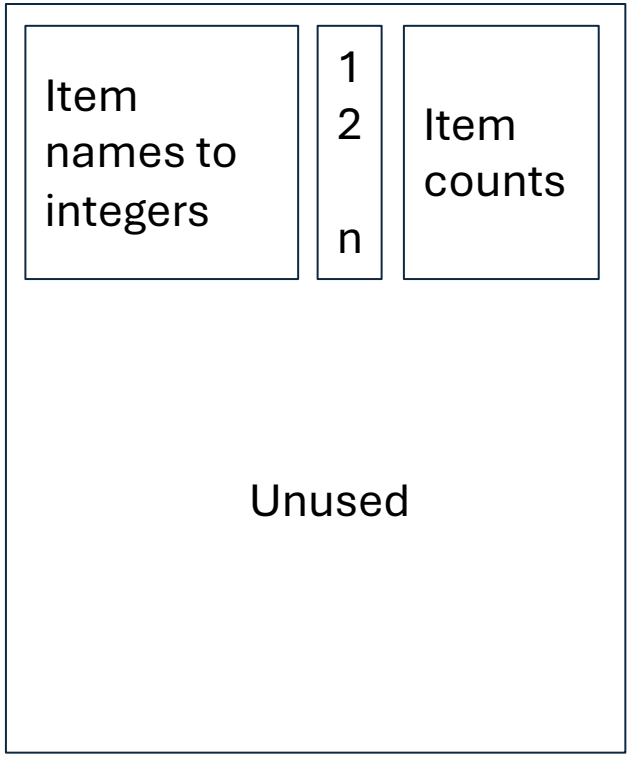
A-Priori Algorithm

- **Pass 1:** Read baskets and count # of occurrences of each item.
  - Requires only memory proportional to # of items.
- **Mark** the items that appear $\geq s$ times as frequent items.
- **Pass 2:** Read baskets keeping track of only the frequent items.
  - Count only those pairs where both elements are frequent.
  - Requires memory proportional to square of # of frequent items.
  - Plus a list of the frequent items (so you know what must be counted).

A-Priori Algorithm (Pass 1)

- **Table 1:** Translate item names to $1, \ldots, n$.
- **Table 2:** Initialize an array of counts to 0.

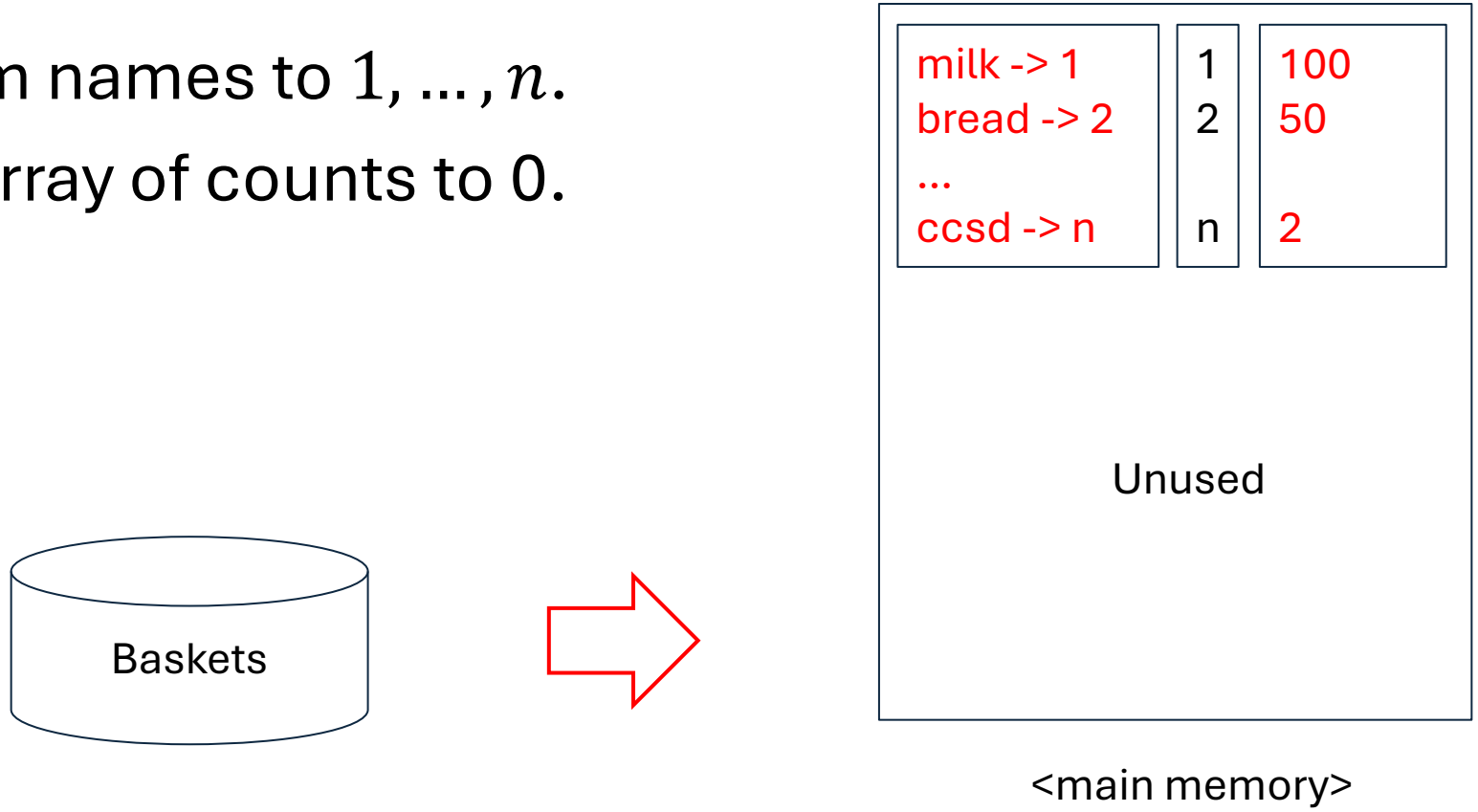




<main memory>

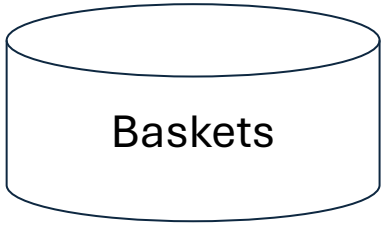
A-Priori Algorithm (Pass 1)

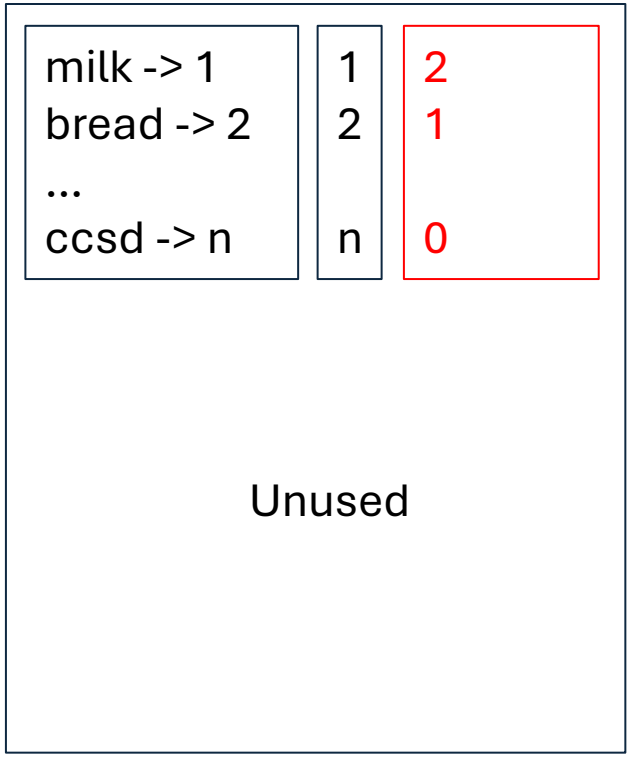
- **Table 1:** Translate item names to $1, \ldots, n$.
- **Table 2:** Initialize an array of counts to 0.



A-Priori Algorithm (between Pass 1 & 2)

- Find frequent items with support $\geq s$.
- Create new numbering $1, \ldots, m$ for the items.
- Create a table whose i-th entry is an integer in $[1, m]$ if item i is frequent or 0 otherwise.

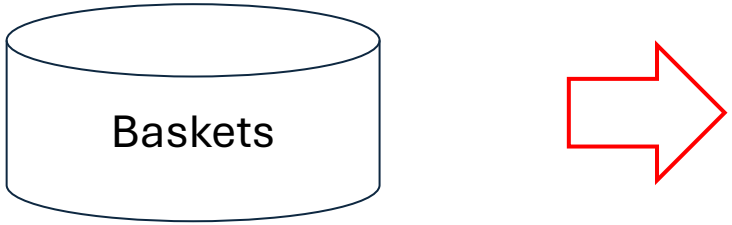


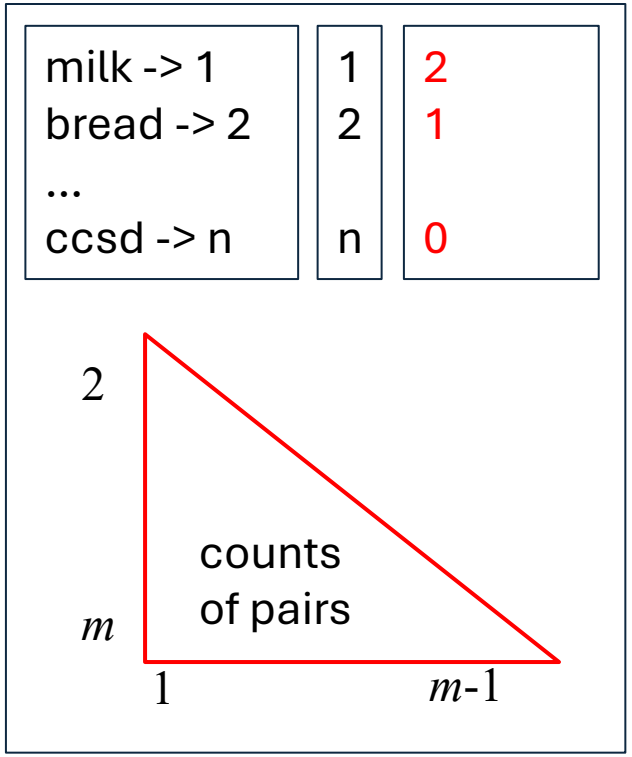


<main memory>

A-Priori Algorithm (Pass 2)

- For each basket, see which items are frequent.
- Generate all pairs of these frequent items.
- For each such pair, increment count in the data structure (triangular matrix or triples).

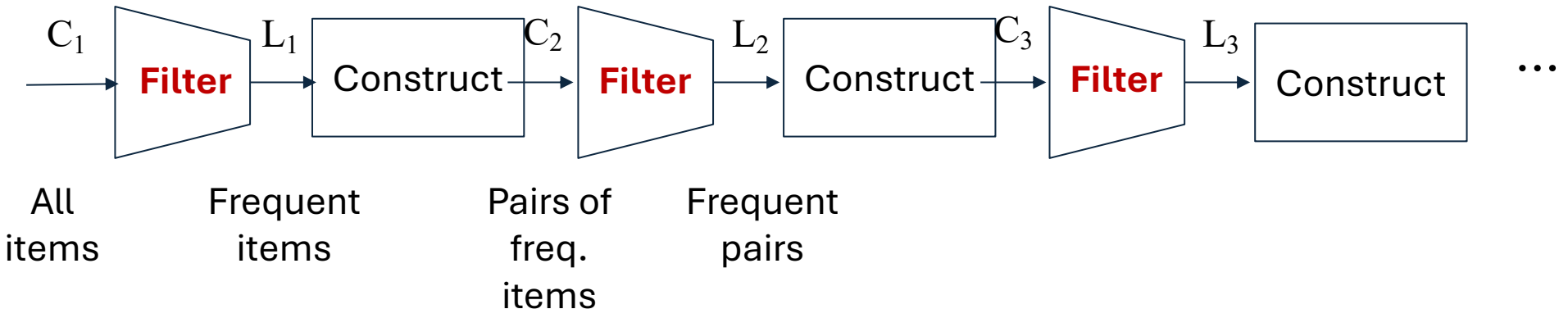




<main memory>

Frequent Triples

- Same technique is used for finding larger frequent itemsets.
- When we move from size $k-1$ to k, we use two sets of itemsets:
  - C_k: Set of candidate k-tuples that might be frequent sets.
  - L_k: Set of truly frequent k-tuples.
- If no frequent itemsets are found, we are done by monotonicity.



Example

- C1 = { {1} {2} {3} {4} {5} {6} {7} {8} {9} {10} }.
- Count the support of itemsets in C1.
- Prune non-frequent itemsets: L1 = { {1}, {2}, {3}, {4}, {5} }.

Example

- C1 = { {1} {2} {3} {4} {5} {6} {7} {8} {9} {10} }.
- Count the support of itemsets in C1.
- Prune non-frequent itemsets: L1 = { {1}, {2}, {3}, {4}, {5} }.
- C2 = { {1,2} {1,3} {1,4} {1,5} {2,3} {2,4} {2,5} {3,4} {3,5} {4,5} }.
- Count the support of itemsets in C2.
- Prune non-frequent itemsets: L2 = { {1,2} {2,3} {2,4} {3,4} {4,5} }.

Example

- C1 = { {1} {2} {3} {4} {5} {6} {7} {8} {9} {10} }.
- Count the support of itemsets in C1.
- Prune non-frequent itemsets: L1 = { {1}, {2}, {3}, {4}, {5} }.
- C2 = { {1,2} {1,3} {1,4} {1,5} {2,3} {2,4} {2,5} {3,4} {3,5} {4,5} }.
- Count the support of itemsets in C2.
- Prune non-frequent itemsets: L2 = { {1,2} {2,3} {2,4} {3,4} {4,5} }.
- Generate C3 = { {2,3,4} } … No {1, 2, 3} or {2, 4, 5}!
- …

Pop Quiz

- Apply the A-Priori algorithm with support threshold $s = 3$:

  B1 = {apple, butter, milk}

  B2 = {apple, butter, banana, walnuts, milk}

  B3 = {butter, milk}

  B4 = {apple, milk}

Summary

1. Frequent Itemsets
   - Market-basket model
2. Association Rules
   - Confidence, Interest, Maximal or closed itemsets
3. Finding Frequent Pairs
   - Triangular-matrix method, Triples method
4. A-Priori Algorithm
   - How the two-pass algorithm works